

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

Facultad de Ingeniería de Sistemas e Informática



CURSO: Lenguajes y Compiladores

DOCENTE: Prudencio Vidal, Javier Antonio

GRUPO: 5

INTEGRANTES:

Arboleda Sanchez, Ericka Sofia	24200047
Badillo Castillo, Edwin Piero	24200150
Ccoicca Fernandez, Adrian	24200154
Huamanchahua Huayanay, Oscar	24200161
Maque Espinoza, Steveen Dennys	24200059
Reynoso Castillo, José Luis	24200169
Zarate Vilchez, Axhell Jesus	24200177

Índice

Índice.....	2
1. Gramática libre de contexto del lenguaje Dummy.....	3
2. Identificación de la recursión izquierda.....	5
3. Eliminación de la recursión izquierda.....	5
3.1. Aplicación al no terminal expresion.....	6
3.2. Gramática transformada (sin recursión izquierda).....	6
3.3. Gramática completa transformada (para parser top-down).....	7
4. Conjuntos FIRST.....	9
5.1. Rutina Programa().....	11
5.2. Rutina Sentencias().....	11
5.3. Rutina Sentencia().....	11
5.4. Rutina Declaracion().....	12
5.5. Rutina Tipo().....	12
5.6. Rutina Asignacion().....	13
5.7. Rutina Condicional().....	13
5.8. Rutina Ciclo().....	13
5.9. Rutina Impresion().....	14
5.10. Rutina Retorno().....	14
5.11. Rutina Expresion().....	14
5.12. Rutina ExprPrima().....	15
5.13. Rutina Factor().....	15
6. Conclusiones.....	16

1. Gramática libre de contexto del lenguaje Dummy

La siguiente gramática libre de contexto (GLC) define formalmente la sintaxis del lenguaje Dummy. Los símbolos en mayúsculas representan terminales (tokens producidos por el lexer) y los símbolos en minúsculas representan no terminales.

1.1. Producción inicial

```
programa -> KW_INICIO DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER
```

1.2. Sentencias

```
sentencias -> sentencia sentencias  
           | /* vacío */
```

```
sentencia -> declaracion DELIM_PUNTO_COMA  
           | asignacion DELIM_PUNTO_COMA  
           | condicional  
           | ciclo  
           | impresion DELIM_PUNTO_COMA  
           | retorno DELIM_PUNTO_COMA
```

1.3. Declaraciones y asignaciones

```
declaracion -> tipo IDENTIFICADOR OP_ASIGNACION expresion  
            | tipo IDENTIFICADOR
```

```
tipo -> KW_NUM | KW_DEC | KW_TEXT
```

```
asignacion -> IDENTIFICADOR OP_ASIGNACION expresion
```

1.4. Estructuras de control

```
condicional -> KW_CUANDO DELIM_PAR_IZQ condicion DELIM_PAR_DER  
              DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER  
            | KW_CUANDO DELIM_PAR_IZQ condicion DELIM_PAR_DER  
              DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER  
            KW_SINO DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER
```

```
ciclo -> KW_LOOP DELIM_PAR_IZQ condicion DELIM_PAR_DER  
        DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER
```

1.5. Impresión y retorno

`impresion` `-> KW_IMPRIMIR DELIM_PAR_IZQ expresion DELIM_PAR_DER`

`retorno` `-> KW_RETORNAR expresion`
 `| KW_RETORNAR`

1.6. Condición

`condicion` `-> expresion operador_rel expresion`
 `| OP_NOT condicion`
 `| condicion OP_AND condicion`
 `| condicion OP_OR condicion`
 `| LIT_BOOLEANO`

`operador_rel` `-> OP_MAYOR | OP_MENOR | OP_MAYOR_IGUAL`
 `| OP_MENOR_IGUAL | OP_IGUAL | OP_DIFERENTE`

1.7. Expresión (con recursión izquierda)

`expresion` `-> expresion OP_OR expresion`
 `| expresion OP_AND expresion`
 `| expresion OP_IGUAL expresion`
 `| expresion OP_DIFERENTE expresion`
 `| expresion OP_MAYOR expresion`
 `| expresion OP_MENOR expresion`
 `| expresion OP_MAYOR_IGUAL expresion`
 `| expresion OP_MENOR_IGUAL expresion`
 `| expresion OP_SUMA expresion`
 `| expresion OP_RESTA expresion`
 `| expresion OP_MULT expresion`
 `| expresion OP_DIV expresion`
 `| expresion OP_MOD expresion`
 `| OP_NOT expresion`
 `| OP_RESTA expresion`
 `| DELIM_PAR_IZQ expresion DELIM_PAR_DER`
 `| factor`

`factor` `-> IDENTIFICADOR | LIT_ENTERO | LIT_DECIMAL`
 `| LIT_TEXT | LIT_BOOLEANO`

2. Identificación de la recursión izquierda

Un parser top-down no puede manejar gramáticas con recursión izquierda. Formalmente, una gramática tiene recursión izquierda si existe un no terminal A tal que $A \Rightarrow^+ A \alpha$ para alguna cadena α . Esto provoca que el parser entre en un bucle infinito sin consumir entrada. En la gramática del lenguaje Dummy, el no terminal `expresion` presenta recursión izquierda directa e inmediata en las siguientes producciones:

```
expresion -> expresion OP_OR expresion
expresion -> expresion OP_AND expresion
expresion -> expresion OP_IGUAL expresion
expresion -> expresion OP_DIFERENTE expresion
expresion -> expresion OP_MAYOR expresion
expresion -> expresion OP_MENOR expresion
expresion -> expresion OP_MAYOR_IGUAL expresion
expresion -> expresion OP_MENOR_IGUAL expresion
expresion -> expresion OP_SUMA expresion
expresion -> expresion OP_RESTA expresion
expresion -> expresion OP_MULT expresion
expresion -> expresion OP_DIV expresion
expresion -> expresion OP_MOD expresion
```

Adicionalmente, el no terminal `sentencias` presenta recursión, pero es recursión por la derecha (`sentencias -> sentencia sentencias`), por lo que no representa un problema para el parser top-down.

3. Eliminación de la recursión izquierda

Se aplica la transformación estándar. Para un no terminal de la forma:

$$A \rightarrow A \alpha_1 \mid A \alpha_2 \mid \dots \mid A \alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$$

Se reescribe como:

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A' \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \varepsilon \end{aligned}$$

donde A' es un nuevo no terminal introducido por la transformación, y ε representa la producción vacía.

3.1. Aplicación al no terminal *expresion*

Para expresion, identificamos los fragmentos:

Tipo	Fragmentos β (bases, sin recursión izquierda)	Fragmentos α (colas recursivas)
Bases (β)	OP_NOT expresion, OP_RESTA expresion, DELIM_PAR_IZQ expresion DELIM_PAR_DER, factor	-
Colas (α)	-	OP_OR expresion, OP_AND expresion, OP_IGUAL expresion, OP_DIFERENTE expresion, OP_MAYOR expresion, OP_MENOR expresion, OP_MAYOR_IGUAL expresion, OP_MENOR_IGUAL expresion, OP_SUMA expresion, OP_RESTA expresion, OP_MULT expresion, OP_DIV expresion, OP_MOD expresion

3.2. Gramática transformada (sin recursión izquierda)

Resultado de la transformación, con solo recursión por la derecha:

```
expresion  -> OP_NOT expresion  expr_prima
            |  OP_RESTA expresion  expr_prima
            |  DELIM_PAR_IZQ expresion DELIM_PAR_DER  expr_prima
            |  factor  expr_prima

expr_prima -> OP_OR expresion  expr_prima
            |  OP_AND expresion  expr_prima
            |  OP_IGUAL expresion  expr_prima
            |  OP_DIFERENTE expresion  expr_prima
            |  OP_MAYOR expresion  expr_prima
            |  OP_MENOR expresion  expr_prima
            |  OP_MAYOR_IGUAL expresion  expr_prima
            |  OP_MENOR_IGUAL expresion  expr_prima
            |  OP_SUMA expresion  expr_prima
            |  OP_RESTA expresion  expr_prima
```

```

|   OP_MULT expresion  expr_prima
|   OP_DIV expresion  expr_prima
|   OP_MOD expresion  expr_prima
|   ε

```

3.3. Gramática completa transformada (para parser top-down)

#	No Terminal	Producción
0	programa	KW_INICIO DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER
1	sentencias	sentencia sentencias
2	sentencias	ε
3	sentencia	declaracion DELIM_PUNTO_COMA
4	sentencia	asignacion DELIM_PUNTO_COMA
5	sentencia	condicional
6	sentencia	ciclo
7	sentencia	impresion DELIM_PUNTO_COMA
8	sentencia	retorno DELIM_PUNTO_COMA
9	declaracion	tipo IDENTIFICADOR OP_ASIGNACION expresion
10	declaracion	tipo IDENTIFICADOR
11	tipo	KW_NUM KW_DEC KW_TEXT
12	asignacion	IDENTIFICADOR OP_ASIGNACION expresion
13	condicional	KW_CUANDO (condicion) { sentencias }
14	condicional	KW_CUANDO (condicion) { sentencias } KW_SINO { sentencias }
15	ciclo	KW_LOOP (condicion) { sentencias }
16	impresion	KW_IMPRIMIR (expresion)
17	retorno	KW_RETORNAR expresion

18	retorno	KW_RETORNAR
19	condicion	expresion operador_rel expresion
20	condicion	OP_NOT condicion
21	condicion	condicion OP_AND condicion
22	condicion	condicion OP_OR condicion
23	condicion	LIT_BOOLEANO
24	operador_rel	OP_MAYOR OP_MENOR OP_MAYOR_IGUAL OP_MENOR_IGUAL OP_IGUAL OP_DIFERENTE
25	expresion	OP_NOT expresion expr_prima
26	expresion	OP_RESTA expresion expr_prima
27	expresion	DELIM_PAR_IZQ expresion DELIM_PAR_DER expr_prima
28	expresion	factor expr_prima
29	expr_prima	OP_OR expresion expr_prima
30	expr_prima	OP_AND expresion expr_prima
31	expr_prima	OP_IGUAL expresion expr_prima
32	expr_prima	OP_DIFERENTE expresion expr_prima
33	expr_prima	OP_MAYOR expresion expr_prima
34	expr_prima	OP_MENOR expresion expr_prima
35	expr_prima	OP_MAYOR_IGUAL expresion expr_prima
36	expr_prima	OP_MENOR_IGUAL expresion expr_prima
37	expr_prima	OP_SUMA expresion expr_prima
38	expr_prima	OP_RESTA expresion expr_prima
39	expr_prima	OP_MULT expresion expr_prima
40	expr_prima	OP_DIV expresion expr_prima
41	expr_prima	OP_MOD expresion expr_prima

42	expr_prima	ϵ
43	factor	IDENTIFICADOR LIT_ENTERO LIT_DECIMAL LIT_TEXT LIT_BOOLEANO

4. Conjuntos FIRST

El conjunto $FIRST(\alpha)$ contiene todos los tokens que pueden aparecer como primer símbolo en alguna cadena derivada de α . Estos conjuntos guían al parser para elegir la producción correcta sin backtracking.

No Terminal	FIRST
programa	{ KW_INICIO }
sentencias	{ KW_NUM, KW_DEC, KW_TEXT, IDENTIFICADOR, KW_CUANDO, KW_LOOP, KW_IMPRIMIR, KW_RETORNAR, ϵ }
sentencia	{ KW_NUM, KW_DEC, KW_TEXT, IDENTIFICADOR, KW_CUANDO, KW_LOOP, KW_IMPRIMIR, KW_RETORNAR }
declaracion	{ KW_NUM, KW_DEC, KW_TEXT }
tipo	{ KW_NUM, KW_DEC, KW_TEXT }
asignacion	{ IDENTIFICADOR }
condicional	{ KW_CUANDO }
ciclo	{ KW_LOOP }
impresion	{ KW_IMPRIMIR }
retorno	{ KW_RETORNAR }
condicion	{ OP_NOT, IDENTIFICADOR, LIT_ENTERO, LIT_DECIMAL, LIT_TEXT, LIT_BOOLEANO, DELIM_PAR_IZQ, OP_RESTA }
expresion	{ OP_NOT, OP_RESTA, DELIM_PAR_IZQ, IDENTIFICADOR, LIT_ENTERO, LIT_DECIMAL, LIT_TEXT, LIT_BOOLEANO }

expr_prima	{ OP_OR, OP_AND, OP_IGUAL, OP_DIFERENTE, OP_MAYOR, OP_MENOR, OP_MAYOR_IGUAL, OP_MENOR_IGUAL, OP_SUMA, OP_RESTA, OP_MULT, OP_DIV, OP_MOD, ϵ }
factor	{ IDENTIFICADOR, LIT_ENTERO, LIT_DECIMAL, LIT_TEXT, LIT_BOOLEANO }

5. Parser Top-Down: Descenso recursivo

Dado que los conjuntos FIRST de las alternativas de cada no terminal son disjuntos, la gramática transformada admite parsing predictivo. Se implementa como un conjunto de rutinas mutuamente recursivas, una por cada no terminal. Cada rutina inspecciona el token actual (lookahead de 1 símbolo) y elige la producción correspondiente sin necesidad de backtracking. Las rutinas del parser son:

Rutina	No Terminal que reconoce
Programa()	programa
Sentencias()	sentencias
Sentencia()	sentencia
Declaracion()	declaracion
Tipo()	tipo
Asignacion()	asignacion
Condicional()	condicional
Ciclo()	ciclo
Impresion()	impresion
Retorno()	retorno
Condicion()	condicion
Expresion()	expresion

ExprPrima()	expr_prima
Factor()	factor

5.1. Rutina Programa()

```
Programa()
  token <- next_token()
  si (token = KW_INICIO) entonces
    token <- next_token()
    si (token = DELIM_LLAVE_IZQ) entonces
      token <- next_token()
      si (Sentencias() = true) entonces
        si (token = DELIM_LLAVE_DER) entonces
          token <- next_token()
          si (token = EOF) entonces
            retornar true // EXITO
          reportar error sintáctico
        retornar false
```

5.2. Rutina Sentencias()

```
Sentencias()
  si (token IN FIRST(sentencia)) entonces
    si (Sentencia() = false) entonces retornar false
    retornar Sentencias() // recursión derecha
  sino // token IN FOLLOW(sentencias): producción vacía
    retornar true
```

5.3. Rutina Sentencia()

```
Sentencia()
  si (token IN {KW_NUM, KW_DEC, KW_TEXT}) entonces
    si (Declaracion() = false) entonces retornar false
    si (token = DELIM_PUNTO_COMA) entonces
      token <- next_token()
      retornar true
  sino si (token = IDENTIFICADOR) entonces
    si (Asignacion() = false) entonces retornar false
    si (token = DELIM_PUNTO_COMA) entonces
      token <- next_token()
      retornar true
  sino si (token = KW_CUANDO) entonces
    retornar Condicional()
```

```

sino si (token = KW_LOOP) entonces
    retornar Ciclo()
sino si (token = KW_IMPRIMIR) entonces
    si (Impresion() = false) entonces retornar false
    si (token = DELIM_PUNTO_COMA) entonces
        token <- next_token()
        retornar true
sino si (token = KW_RETORNAR) entonces
    si (Retorno() = false) entonces retornar false
    si (token = DELIM_PUNTO_COMA) entonces
        token <- next_token()
        retornar true
reportar error sintáctico
retornar false

```

5.4. Rutina Declaracion()

```

Declaracion()
    si (Tipo() = false) entonces retornar false
    si (token = IDENTIFICADOR) entonces
        token <- next_token()
        si (token = OP_ASIGNACION) entonces // declaracion con
inicialización
            token <- next_token()
            retornar Expresion()
        retornar true // declaracion simple sin inicialización
    reportar error sintáctico
    retornar false

```

5.5. Rutina Tipo()

```

Tipo()
    si (token = KW_NUM) entonces
        token <- next_token()
        retornar true
    sino si (token = KW_DEC) entonces
        token <- next_token()
        retornar true
    sino si (token = KW_TEXT) entonces
        token <- next_token()
        retornar true
    reportar error sintáctico
    retornar false

```

5.6. Rutina Asignacion()

```
Asignacion()  
  si (token = IDENTIFICADOR) entonces  
    token <- next_token()  
    si (token = OP_ASIGNACION) entonces  
      token <- next_token()  
      retornar Expresion()  
  reportar error sintáctico  
  retornar false
```

5.7. Rutina Condicional()

```
Condicional()  
  si (token = KW_CUANDO) entonces  
    token <- next_token()  
    si (token = DELIM_PAR_IZQ) entonces  
      token <- next_token()  
      si (Condicion() = true) entonces  
        si (token = DELIM_PAR_DER) entonces  
          token <- next_token()  
          si (token = DELIM_LLAVE_IZQ) entonces  
            token <- next_token()  
            si (Sentencias() = true) entonces  
              si (token = DELIM_LLAVE_DER) entonces  
                token <- next_token()  
                retornar true  
          sino // sin rama sino  
            retornar true  
        reportar error sintáctico  
        retornar false
```

5.8. Rutina Ciclo()

```
Ciclo()  
  si (token = KW_LOOP) entonces  
    token <- next_token()  
    si (token = DELIM_PAR_IZQ) entonces
```

```

token <- next_token()
si (Condicion() = true) entonces
si (token = DELIM_PAR_DER) entonces
token <- next_token()
si (token = DELIM_LLAVE_IZQ) entonces
    token <- next_token()
    si (Sentencias() = true) entonces
    si (token = DELIM_LLAVE_DER) entonces
        token <- next_token()
        retornar true
reportar error sintáctico
retornar false

```

5.9. Rutina Impresion()

```

Impresion()
si (token = KW_IMPRIMIR) entonces
    token <- next_token()
    si (token = DELIM_PAR_IZQ) entonces
        token <- next_token()
        si (Expresion() = true) entonces
        si (token = DELIM_PAR_DER) entonces
            token <- next_token()
            retornar true
reportar error sintáctico
retornar false

```

5.10. Rutina Retorno()

```

Retorno()
si (token = KW_RETORNAR) entonces
    token <- next_token()
    si (token IN FIRST(expresion)) entonces // retorno con valor
        retornar Expresion()
    retornar true // retorno vacio
reportar error sintáctico
retornar false

```

5.11. Rutina Expresion()

Esta rutina implementa la gramática ya transformada (sin recursión izquierda):

```

Expresion()
si (token = OP_NOT) entonces
    token <- next_token()

```

```

        si (Expresion() = false) entonces retornar false
        retornar ExprPrima()
    sino si (token = OP_RESTA) entonces
        token <- next_token()
        si (Expresion() = false) entonces retornar false
        retornar ExprPrima()
    sino si (token = DELIM_PAR_IZQ) entonces
        token <- next_token()
        si (Expresion() = true) entonces
            si (token = DELIM_PAR_DER) entonces
                token <- next_token()
                retornar ExprPrima()
            reportar error sintáctico
            retornar false
    sino si (token IN FIRST(factor)) entonces // IDENTIFICADOR o
literal
        si (Factor() = false) entonces retornar false
        retornar ExprPrima()
    reportar error sintáctico
    retornar false

```

5.12. Rutina ExprPrima()

Maneja las colas de operaciones binarias. La producción vacía (ϵ) se activa cuando el token actual no pertenece a ningún operador binario conocido.

```

ExprPrima()
    si (token IN {OP_OR, OP_AND, OP_IGUAL, OP_DIFERENTE,
        OP_MAYOR, OP_MENOR, OP_MAYOR_IGUAL, OP_MENOR_IGUAL,
        OP_SUMA, OP_RESTA, OP_MULT, OP_DIV, OP_MOD}) entonces
        token <- next_token() // consumir el operador
        si (Expresion() = false) entonces retornar false
        retornar ExprPrima() // recursión derecha
    sino // producción vacía: token IN FOLLOW(expr_prima)
        retornar true

```

5.13. Rutina Factor()

```

Factor()
    si (token = IDENTIFICADOR) entonces
        token <- next_token()
        retornar true
    sino si (token = LIT_ENTERO) entonces
        token <- next_token()
        retornar true

```

```

sino si (token = LIT_DECIMAL) entonces
    token <- next_token()
    retornar true
sino si (token = LIT_TEXT) entonces
    token <- next_token()
    retornar true
sino si (token = LIT_BOOLEANO) entonces
    token <- next_token()
    retornar true
reportar error sintáctico
retornar false

```

6. Conclusiones

La gramática libre de contexto del lenguaje Dummy es una gramática de tipo 2 en la jerarquía de Chomsky. Su no terminal principal expresión contiene recursión izquierda inmediata, lo que la hace incompatible directamente con un parser top-down.

Aplicando la transformación estándar de eliminación de recursión izquierda, se introduce el nuevo no terminal `expr_prima`, obteniendo una gramática equivalente que define el mismo lenguaje pero con recursión únicamente por la derecha.

La gramática transformada admite parsing predictivo (top-down sin backtracking), ya que los conjuntos FIRST de las alternativas de cada no terminal son disjuntos. Esto permite implementar el parser como un conjunto de 15 rutinas mutuamente recursivas (descenso recursivo), donde cada rutina inspecciona el token actual y elige la producción correcta con lookahead de exactamente un símbolo (LL(1)).